Programare web

Laboratorul 10

10. Serverul web. Node.js

Cuprins

10. Serverul web. Node.js 1
10.1. Concepte și tehnologii 1
10.2. Structura proiectului 1
10.3. Descriere 2
10.3.1. Instalarea Node.js, pregătirea proiectului și rularea serverului 2
10.3.2. Managerul de pachete npm 2
10.3.3. Web framework-ul Express 3
10.3.4. Informații suplimentare 4
10.4. Teme 4
10.4.1. Tema 1 4
10.4.2. Tema 2 5
10.4.3. Tema 3 5

10.1. Concepte și tehnologii

În acest laborator veți începe proiectul 2 și veți lucra cu serverul web Node.js. De asemenea, veți lucra în continuare cu sistemul de versionare git și cu aplicația Visual Studio Code.

10.2. Structura proiectului

Temele din laboratoarele 10-13 vor forma un singur proiect: *proiect-2-numeUtilizatorGitHub*.

De data aceasta proiectul va avea o tematică impusă.

Astfel, veți realiza un site web care va permite utilizatorilor să cumpere produse dintr-o anumită categorie sau dintr-o sub-categorie a acesteia.

Pentru a afla categoria de produse (tema site-ului vostru) trebuie să accesați quiz-ul de la adresa https://edu.tuiasi.ro/mod/quiz/view.php?id=45291.

Structura proiectului `proiect-2-numeUtilizatorGitHub` de până la acest laborator (inclusiv):

- [`node_modules`] - conține pachetele necesare proiectului și este creat cu utilitarul npm (nu trebuie pus pe git)
- [`public`] - conține toate resursele accesibile direct de către client (e.g., fișiere css, javascript, imagini)
- [`views`]
  - `chestionar.ejs` - lab 10
  - `layout.ejs` - lab 10
  - `rezultat-chestionar.ejs` - lab 10 (tema 2)
- `.gitignore` - lab 10 (conține doar textul `node_modules`)

- `app.js` - lab 10
- `intrebari.json` - lab 10 (tema 3)
- `package-lock.json` - lab 10 (este generat de npm init sau npm install și ar putea să lipsească)
- `package.json` - lab 10
- `README.md` - lab 10

10.3. Descriere

10.3.1. Instalarea Node.js, pregătirea proiectului și rularea serverului

1. Descărcați și instalați ultima versiune LTS (Long-term support) a Node.js (22.15.0) de la adresa https://nodejs.org/en/download/. Nu este nevoie pe calculatoarele din laborator.
2. Dacă nu ați făcut-o deja, creați proiectul pe GitHub accesând adresa de pe tablă și urmați pașii din secțiunea Începutul laboratorului din laboratorul 0.
3. Deschideți proiectul în Visual Studio Code și creați un fișier cu numele `app.js` în care copiați următorul cod:

```
const http = require('http');
const hostname = '127.0.0.1';
const port = 6789;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World');
});

server.listen(port, hostname, () => {
  console.log(`Serverul rulează la adresa http://${hostname}:${port}/`);
});
```

4. În terminal executați comanda `node app.js` pentru a porni serverul.
5. Testați în browser accesând adresa http://localhost:6789/.

10.3.2. Managerul de pachete npm

1. În terminalul din Visual Studio Code, executați comanda `npm init`.
   Această comandă va crea și configura un fișier `package.json` care conține informații despre proiect și dependențele sale.
2. Pentru a accepta valoarea din paranteză pentru proprietățile respective puteți apăsa tasta ENTER. La proprietatea `test command` nu introduceți nimic.
3. Executați comanda `npm install` pentru a instala cele necesare proiectului prin procesarea fișierului `package.json`.
   Astfel, se va crea un fișier `package-lock.json` care nu ar trebui editat.
4. Executați comanda `npm install -g nodemon` pentru a instala global (-g) aplicația nodemon care permite repornirea automată a aplicației voastre de fiecare dată când se modifică un fișier. Pe calculatoarele din laborator ar trebui să fie deja instalat.
   Astfel, lansați în execuție aplicația prin comanda: `nodemon app.js`.

În Windows 10 va trebui să rulați o comandă în PowerShell: apăsați butonul Windows din taskbar ➜ scrieți PowerShell ➜ click dreapta pe Windows PowerShell ➜ Run as administrator ➜ în terminalul care se deschide executați: `Set-ExecutionPolicy -Scope LocalMachine Unrestricted`.

10.3.3. Web framework-ul Express

1. Executați comanda `npm install express --save` pentru a instala web framework-ul Express care facilitează implementarea unei aplicații web.
   Astfel, se va crea un director `node_modules` în rădăcina proiectului în care nu ar trebui să faceți modificări.
2. Executați comanda `npm install ejs --save` pentru a instala template engine-ul ejs care facilitează crearea resurselor html dinamice.
3. Executați comanda `npm install express-ejs-layouts --save` pentru a adăuga suport pentru layout-uri (template-ul folosit de site-ul vostru).
4. Executați comanda `npm install body-parser --save` pentru a interpreta corpul mesajului cererii.
5. Înlocuiți codul din fișierul `app.js` cu următorul cod:

```
const express = require('express');
const expressLayouts = require('express-ejs-layouts');
const bodyParser = require('body-parser')

const app = express();

const port = 6789;

// directorul 'views' va conține fișierele .ejs (html + js executat la server)
app.set('view engine', 'ejs');
// suport pentru layout-uri - implicit fișierul care reprezintă template-ul site-ului
este views/layout.ejs
app.use(expressLayouts);
// directorul 'public' va conține toate resursele accesibile direct de către client
(e.g., fișiere css, javascript, imagini)
app.use(express.static('public'))
// corpul mesajului poate fi interpretat ca json; datele de la formular se găsesc în
format json în req.body
app.use(bodyParser.json());
// utilizarea unui algoritm de deep parsing care suportă obiecte în obiecte
app.use(bodyParser.urlencoded({ extended: true }));

// la accesarea din browser adresei http://localhost:6789/ se va returna textul 'Hello
World'
// proprietățile obiectului Request - req - https://expressjs.com/en/api.html#req
// proprietățile obiectului Response - res - https://expressjs.com/en/api.html#res
app.get('/', (req, res) => res.send('Hello World'));

// la accesarea din browser adresei http://localhost:6789/chestionar se va apela funcția
specificată
app.get('/chestionar', (req, res) => {
    const listaIntrebari = [
        {
```

```
            intrebare: 'Întrebarea 1',
            variante: ['varianta 1', 'varianta 2', 'varianta 3', 'varianta 4'],
            corect: 0
        },
        //...
    ];
    // în fișierul views/chestionar.ejs este accesibilă variabila 'intrebari' care
conține vectorul de întrebări
    res.render('chestionar', {intrebari: listaIntrebari});
});

app.post('/rezultat-chestionar', (req, res) => {
    console.log(req.body);
    res.send("formular: " + JSON.stringify(req.body));
});

app.listen(port, () => console.log(`Serverul rulează la adresa http://localhost:
:${port}/`));
```

10.3.4. Informații suplimentare

Pentru detalii privind Node.js să citiți cursul 11 și să consultați documentația următoare:

a. Introduction to Node.js - https://nodejs.dev/en/learn/introduction-to-nodejs/
b. Node.js server without a framework - https://developer.mozilla.org/en-US/docs/Learn/Server-side/Node_server_without_framework
c. Express basic routing - https://expressjs.com/en/starter/basic-routing.html
d. Serving static files in Express - https://expressjs.com/en/starter/static-files.html
e. ejs - Embedded JavaScript templates - https://www.npmjs.com/package/ejs
f. Getting started with EJS - https://medium.com/@atingenkay/getting-started-with-ejs-28fe4a693e53
g. express-ejs-layouts - https://www.npmjs.com/package/express-ejs-layouts

10.4. Teme

10.4.1. Tema 1

Urmați pașii prezentați în acest laborator și citiți cu atenție comentariile din cod.

De abia la ultima temă va trebui să faceți ca site-ul să arate frumos. Accentul este acum pe funcționalitate.

Implementați următoarele funcționalități.

a. Creați în directorul `views` un fișier cu numele `layout.ejs` în care puneți scheletul site-ului (template-ul).
   În momentul în care se va apela `res.render`, se va returna conținutul fișierului `layout.ejs` și, unde apare `<%- body %>`, se va insera conținutul view-ului specificat ca argument, (e.g., `chestionar.ejs`).
b. Creați în directorul `views` un fișier cu numele `chestionar.ejs` în care să fie formularul cu quiz-ul respectiv.
   În acest fișier aveți acces la variabila `intrebari` specificată ca argument la apelul `res.render`. Pentru a crea formularul trebuie să parcurgeți vectorul `intrebari` și să adăugați întrebările și variantele de răspuns (va trebui să creați elemente de tipul `<input type="radio" .../>`).
c. Testați în browser accesând adresa http://localhost:6789/chestionar.

10.4.2. Tema 2

Extindeți proiectul prin implementarea următoarelor funcționalități:

a. Adăugați mai multe întrebări cu variantele lor de răspuns referitoare la tema impusă.
b. Adăugați logica de determinare a numărului de răspunsuri corecte. După completarea chestionarului se va face o cerere pentru resursa `/rezultat-chestionar` (vezi atributul `action` a formularului), unde se va afișa informația.
c. Faceți ca resursa `/rezultat-chestionar` să aibă același layout (template) ca resursa `/chestionar`.

10.4.3. Tema 3

Extindeți proiectul astfel încât întrebările afișate în chestionar să fie stocate într-un fișier `intrebari.json`.

Pentru a citi fișierul utilizați varianta **asincronă** prezentată în articolul Reading and Writing JSON Files with Node.js - https://stackabuse.com/reading-and-writing-json-files-with-node-js/.

De asemenea, faceți ca site-ul să arate frumos.